

ChatGPT Response

Yes — I checked the actual Excel file you uploaded, and the error is **most likely happening because of the Excel file format/structure, not because of your lead data itself.**

The error:

Unexpected token 'R', "Request En"... is not valid JSON

usually means the importer expected the server to return a JSON response, but instead the server returned plain text beginning with something like “**Request En...**”. In other words, the frontend is trying to parse an error response as JSON.

I found a problem in your Excel file

Your file is:

MORBI ALL DATA FINAL copy 2.xlsx

When I inspected its internal Excel structure, it contains a worksheet called:

date updated january 2026

but the workbook is using **Strict OOXML / newer Excel XML specifications:**

conformance="strict"

Your workbook also contains some Microsoft-specific/revision metadata.

Most importantly, when I tried to read the workbook normally, the Excel parser reported:

invalid specification for 0

and could not properly load the worksheet.

So your importer may be receiving a workbook that **Excel itself can open, but the application's Excel parser/import API cannot properly parse.**

What your "Lead-Only Import Protocol" means

This part:

Vehicle Registration Numbers and Phone Numbers to verify if leads already exist... automatically updating records rather than creating duplicates

is **not the cause of this JSON error**.

It simply means your application is designed to do this:

Excel data	System action
Vehicle number already exists	Update existing lead
Phone number already exists	Update existing lead
Neither exists	Create new lead
Same vehicle + phone appears multiple times	Should ideally deduplicate/update

So the importer is specifically looking for fields such as:

Vehicle Registration Number
Phone Number

and probably other vehicle/contact columns.

The error is occurring BEFORE that logic can properly run

The likely flow is:

Upload Excel → Backend tries to read Excel → Excel parser fails → Backend sends "Request En..." error → Frontend expects JSON → JSON parsing fails → you see "Unexpected token 'R'"

So don't start changing all your vehicle numbers/phone numbers yet. **First we need to fix the Excel file itself.**

Best solution

I recommend converting this workbook into a **standard .xlsx file** with a clean worksheet, rather than uploading this particular version directly.

If you want, I can take **this exact uploaded Excel file**, clean/rebuild it into a proper importer-ready Excel file while preserving the data, and give you the corrected file to upload.

I can also check the column names against your "Lead-Only Import Protocol" and tell you exactly which columns need to be renamed/added before you upload it.

Supabse answer

I'm checking the project's Edge Functions and the current Supabase guidance for non-JSON upload responses. The error usually means the importer tried to parse a plain-text or HTML error response as JSON.

```
Ran load_knowledge
Ran list_edge_functions
Reasoned
Ran search_docs
Reasoned
```

The `R` strongly indicates the server returned text beginning with “Request Entity Too Large” (HTTP 413), and your frontend then called `.json()` on that text. I'm checking recent function logs to confirm whether the upload was rejected by the Edge Function or by an upstream limit.

```
Ran load_knowledge
Reasoned
```

Failed to query logs.

```
Reasoned
```

What This Error Means

```
Unexpected token 'R', "Request En"... is not valid JSON
```

Your frontend expects a JSON response, but the server returned plain text beginning with:

```
Request Entity Too Large
```

That normally means the uploaded sheet is too large for the request. Your code then tries to run `response.json()` on that text, causing the JSON parsing error.

The “Lead-Only Import Protocol” is not the error. It is only the importer's description of how it should process vehicle registration numbers and phone numbers.

Most Likely Causes

- The Excel/CSV file is too large.
- The file is converted to Base64 before upload, increasing its size.
- The entire sheet is sent inside a JSON request.
- The Edge Function or gateway rejects the request before the function runs.
- The frontend assumes every response is JSON.

Fix the Frontend Error Handling

Do not parse every response directly with `.json()`:

```
const response = await fetch(uploadUrl, {

  method: 'POST',

  body: formData,

})

const responseText = await response.text()

let result: unknown

try {

  result = responseText ? JSON.parse(responseText) : null

} catch {

  throw new Error(

    response.status === 413

      ? 'The sheet is too large. Please upload a smaller file.'

      : responseText || `Upload failed with status ${response.status}`

  )

}

if (!response.ok) {

  throw new Error(

    typeof result === 'object' && result !== null && 'error' in result

      ? String((result as { error: unknown }).error)

      : `Upload failed with status ${response.status}`

  )

}
```

```
}
```

If using `supabase.functions.invoke()`:

```
const { data, error } = await
supabase.functions.invoke('your-import-function', {

  body: formData,

})

if (error) {

  if ('context' in error && error.context instanceof Response) {

    const text = await error.context.text()

    throw new Error(

      error.context.status === 413

        ? 'The sheet is too large. Please upload a smaller file.'

        : text || error.message

    )

  }

  throw new Error(error.message)

}
```

Use `FormData` for the Sheet

Do not send the spreadsheet as a large JSON string or Base64 value:

```
const formData = new FormData()

formData.append('file', selectedFile)

const response = await fetch(

  `${supabaseUrl}/functions/v1/your-import-function`,
```

```

{
  method: 'POST',

  headers: {

    Authorization: `Bearer ${session.access_token}`,

  },

  body: formData,

}

)

```

Do not manually set this header when using `FormData`:

```
'Content-Type': 'application/json'
```

The browser must set the multipart boundary automatically.

Recommended Import Design

For large sheets:

- Upload the file to a private Supabase Storage bucket.
- Send only the Storage file path to the Edge Function.
- Let the function process rows in batches.
- Insert or update leads in batches using the vehicle registration number and phone number.
- Return a JSON summary such as imported, updated, skipped, and failed rows.

For an immediate test, try a much smaller sheet. If the smaller file works, the cause is confirmed as a request-size limit.

The project logs could not be checked because log access is also disabled by the organization's data-sharing settings. To identify the exact failing line, I would need the client upload code and the Edge Function code for this importer.

Supabase properly answer

I'm checking the project's Edge Functions and the current Supabase guidance for non-JSON upload responses. The error usually means the importer tried to parse a plain-text or HTML error response as JSON.

```
Ran load_knowledge
Ran list_edge_functions
Reasoned
Ran search_docs
Reasoned
```

The error strongly indicates that the server returned text beginning with “**Request Entity Too Large**” (HTTP 413), and your frontend then called `.json()` on that text. I'm checking recent function logs to confirm whether the upload was rejected by the Edge Function or by an upstream limit.

```
Ran load_knowledge
Reasoned
Failed to query logs.
Reasoned
```

What This Error Means

Error:

```
Unexpected token 'R', "Request En"... is not valid JSON
```

Your frontend expects a JSON response, but the server returned plain text beginning with:

```
Request Entity Too Large
```

That normally means the uploaded sheet is too large for the request. Your code then tries to run `response.json()` on that text, which causes the JSON parsing error.

The “**Lead-Only Import Protocol**” is not the error. It is only the importer's description of how it should process vehicle registration numbers and phone numbers.

Most Likely Causes

- The Excel/CSV file is too large.
- The file is converted to Base64 before upload, increasing its size.
- The entire sheet is sent inside a JSON request.

- The Edge Function or gateway rejects the request before the function runs.
- The frontend assumes every response is JSON.

Fix the Frontend Error Handling

Do not parse every response directly with `.json()`:

```
const response = await fetch(uploadUrl, {
  method: 'POST',
  body: formData,
})

const responseText = await response.text()

let result: unknown

try {
  result = responseText ? JSON.parse(responseText) : null
} catch {
  throw new Error(
    response.status === 413
      ? 'The sheet is too large. Please upload a smaller file.'
      : responseText || `Upload failed with status ${response.status}`
  )
}

if (!response.ok) {
  throw new Error(
    typeof result === 'object' &&
    result !== null &&
    'error' in result
      ? String((result as { error: unknown }).error)
      : `Upload failed with status ${response.status}`
  )
}
```

If Using `supabase.functions.invoke()`

```
const { data, error } = await supabase.functions.invoke(
  'your-import-function',
  {
    body: formData,
  }
)
```

```

    }
  )

  if (error) {
    if ('context' in error && error.context instanceof Response) {
      const text = await error.context.text()

      throw new Error(
        error.context.status === 413
          ? 'The sheet is too large. Please upload a smaller file.'
          : text || error.message
      )
    }

    throw new Error(error.message)
  }

```

Use FormData for the Sheet

Do not send the spreadsheet as a large JSON string or Base64 value:

```

const formData = new FormData()

formData.append('file', selectedFile)

const response = await fetch(
  `${supabaseUrl}/functions/v1/your-import-function`,
  {
    method: 'POST',
    headers: {
      Authorization: `Bearer ${session.access_token}`,
    },
    body: formData,
  }
)

```

When using **FormData**, do not manually set the **Content-Type** header:

```
'Content-Type': 'application/json'
```

The browser must set the **multipart/form-data** boundary automatically.

Recommended Import Design

For large sheets:

1. Upload the file to a private Supabase Storage bucket.
2. Send only the Storage file path to the Edge Function.
3. Let the function process the rows in batches.
4. Insert or update leads in batches using the vehicle registration number and phone number.
5. Return a JSON summary such as `imported`, `updated`, `skipped`, and `failed` rows.

For an immediate test, try uploading a much smaller sheet. If the smaller file works, that confirms that the issue is related to a request-size limit.

The project logs could not be checked because log access is also disabled by the organization's data-sharing settings. To identify the exact failing line, I would need the client upload code and the Edge Function code for this importer.